KLH Logo

SYSTEM CALL MONITOR

Team Id : 13

Section: 9 Members: G . SAHITHI - 2520030499 , B . JASMITHA - 2520030557

Guide: Y . HARIKA Operating Systems and Systems Programming (OSSP) | KLH Bachupally Campus, CSE | PBL 2026–27

Abstract

The System Call Demonstration and Monitoring System is designed to demonstrate how an operating system interacts with hardware and system resources through system calls. It provides practical demonstrations of file operations, process management, inter-process communication, memory management, file-system operations and concurrency using C programming in Linux.

Problem Statement & Objectives

Understanding system calls theoretically can be difficult without practical demonstrations of how operating systems manage processes, files, memory and resources.
Demonstrate commonly used Linux system calls.
Understand process creation and execution.
Explore IPC and signal handling.
Demonstrate memory and file-system operations.

Existing vs Proposed System

Aspect	Basic / Existing Approach	Proposed Project
Process handling	Mainly theoretical learning	Practical demonstration using fork() , exec() and waitpid()
Communication	Limited process communication	Demonstrates pipes and signals
Resource handling	Limited file and meomry operations	Demonstrates file and memory management
Error handling	Errors maybe difficult to identify	Uses error checking and validation
Key gap	Lack of practical system call demonstrations	Provides hands on learning through a single c- based system

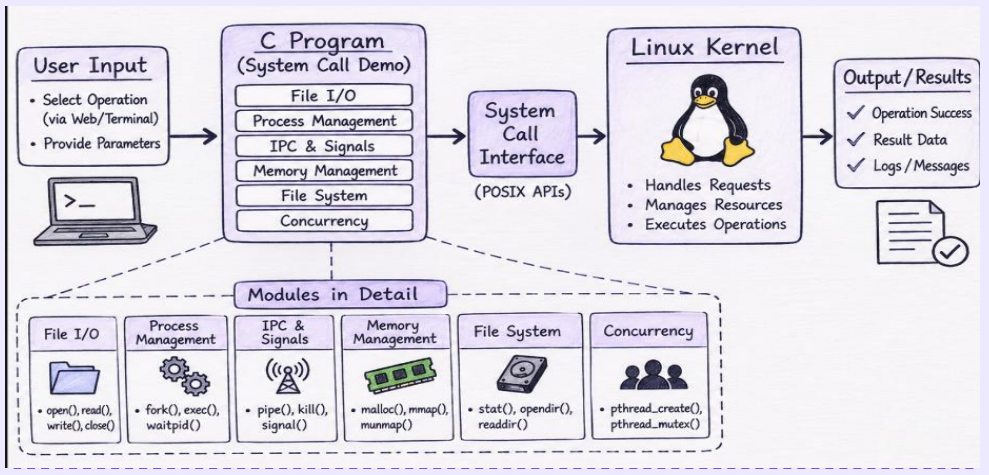
Feasibility Study

- Technical: Uses C, Linux, GCC and system calls to demonstrate OS operations.
- Economic: Uses free tools like Ubuntu and GCC, with no extra cost.
- Operational: Easy to use through the Linux terminal for practical demonstrations.
- Schedule: Includes planning, development and testing within the semester.

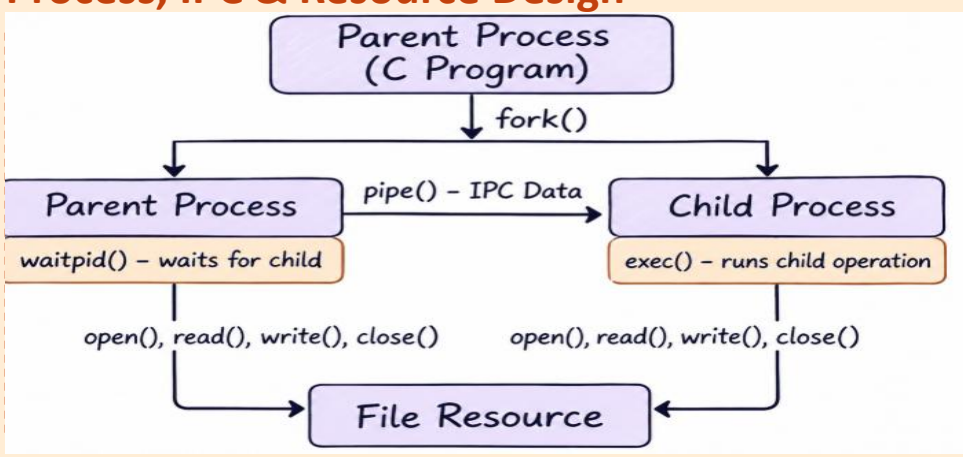
Tools & Technologies

- Programming Language: C
- Operating System: Ubuntu Linux
- Compiler: GCC
- System Interface: POSIX APIs
- Libraries: pthread, signal, unistd, sys/types
- Development Environment: Ubuntu Terminal

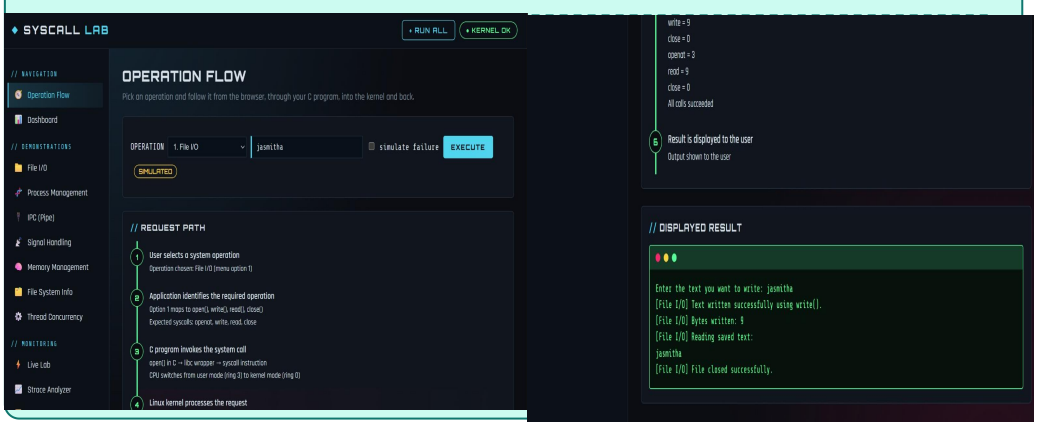
System Workflow / Architecture



Process, IPC & Resource Design



Implementation Screenshots



Results & Testing

- Successfully demonstrated file operations, process creation and communication using system callsTested normal operations, invalid inputs, error handling and process execution.

Conclusion

- The project demonstrates how C programs interact with the Linux kernel through system calls.

It provides practical understanding of file handling, process management, IPC and other OS concepts.

Future Scope

- Add more Linux system calls and advanced OS features.
- Improve monitoring, performance analysis and error reporting

References: [1] Linux man-pages [2] Operating Systems textbook / course material [3] GCC / GNU documentation

QR: GitHub